

Autonomous Credit Intelligence: An Event-Driven, Self-Healing Multi-Agent Framework for Real-Time B2B Risk Assessment

Nikhil Nagendra, Pallavi K V, Nitin Bharadwaj MVS, Johan K George, Prasanna B Acharya, Bhoomi Joshi
Department of Computer Science and Engineering
AMC Engineering College
Bengaluru 560083, Karnataka, India
Email: nikhilnag98@gmail.com

Abstract—Batch reviews of B2B accounts receivable often miss intraday credit deterioration because payment behaviour, financial fundamentals, and litigation signals are evaluated in disconnected, periodic cycles. This paper presents ACIS-X (Autonomous Credit Intelligence System, Extended), an event-driven multi-agent framework on Apache Kafka that continuously maintains a live per-customer risk score. Specialised agents coordinate exclusively through Kafka events across five core processing layers, an autonomous model governance plane, and a cross-cutting self-healing plane. The pipeline features native Kafka Exactly-Once Semantics (EOS), three-tier signal fusion, SHAP audit persistence, statistical drift tracking via Population Stability Index (PSI), online challenger model benchmarking, and heartbeat-driven recovery from committed offsets. To evaluate the architecture under diverse macroeconomic stress cases, we utilize a jump-diffusion synthetic stress test harness. The evaluation demonstrates real-time streaming throughput, horizontal consumer group scalability, autonomous fault recovery with storm prevention, and the contribution of behavioural refinement and multi-signal fusion under controlled synthetic evaluation.

Index Terms—Multi-agent systems, Event-driven architecture, Credit risk intelligence, Apache Kafka, Exactly-Once Semantics, Self-healing systems.

I. Introduction

Management of enterprise credit risks calls for constant evaluation of payment status, financial position, and potential litigations. Daily or weekly batch cycles delay action when risk rises within a day. Event streaming platforms such as Apache Kafka [1] and emerging multi-agent designs [2], [11] make it practical to maintain a live credit view instead of an overnight refresh. However, while recent deliberative LLM-based multi-agent systems [2], [12] offer qualitative reasoning, their multi-second latency and non-deterministic outputs make them unsuitable for high-throughput, sub-50 ms financial streaming pipelines subject to strict operational risk controls.

This work introduces ACIS-X (Autonomous Credit Intelligence System, Extended), a Kafka-based reactive, event-driven multi-agent framework (ED-MAS) grounded in the Actor model for real-time B2B credit intelligence. It targets three gaps in prior practice: (1) latency of batch monitoring; (2) siloed behavioural, financial, and litigation

signals; and (3) fragile pipelines without autonomous recovery and drift governance.

The contributions are: a layered Kafka-only reactive agent architecture with a cross-cutting self-healing plane and autonomous model governance; end-to-end Kafka Exactly-Once Semantics (EOS); a three-tier fusion path combining behavioural metrics, external financials, and litigation into one score in $[0,1]$ backed by regulatory policy guardrails; an autonomous governance agent supporting model-risk governance principles aligned with SR 11-7 via PSI drift tracking and challenger model benchmarking; heartbeat monitoring with deterministic restart from committed offsets; and a comprehensive test suite measuring latency, deduplication, scaling, predictive quality, weight sensitivity, and fault recovery under a jump-diffusion synthetic stress test harness.

II. Related Work

Traditional credit scoring methods like Altman’s Z-score [3] and more recently explainable ML methods for corporate credit [4], [5] all involve offline generated or periodically updated feature tables, making them less suited to intraday monitoring although SHAP [6] makes these approaches more interpretable. Ensemble or hybrid approaches provide better prediction results [5], yet they remain batch learning methods and do not process invoice streams in real time. Event sourcing and processing pipelines based on Kafka can be used for financial fraud analytics with sub-second latency [7], [8], although they focus mainly on transaction-level scoring, not on B2B credit profiling which combines financial fundamentals with litigation data.

The multi-agent approach to credit scoring is gaining attention too. For instance, MASCA [2], Kubam’s agentic framework [11], and CreditAgent [12] study the use of LLM reasoning and MARL for credit evaluation. Nevertheless, these solutions are focused more on deliberative reasoning than on high-throughput event stream processing. Recent stream processing microservice solutions incorporate circuit breakers and replay mechanisms [10],

yet usually restore processing after failover by restarting entire containers, not offering more granular per-agent failover through heartbeat-based failure detection.

Table I contrasts the operational and architectural characteristics of prior paradigms against ACIS-X.

As shown in Table I, ACIS-X uniquely occupies the intersection of streaming credit fusion, deterministic compliance, autonomous governance, and fine-grained agent recovery: specialized reactive agents exchange only Kafka events, fuse three signal families online, and resume autonomously from the last committed offset.

III. System Architecture

ACIS-X maintains a live risk score for each customer c_i :

$$\mathcal{R}(c_i, t) = \mathcal{F}(\mathcal{B}(c_i, t), \mathcal{E}(c_i, t), \mathcal{L}(c_i, t)) \in [0, 1], \quad (1)$$

where $\mathcal{B}$ is behavioural history, $\mathcal{E}$ external financial indicators, and $\mathcal{L}$ litigation exposure. Design targets are sub-second processing, usable scores under partial signals, zero silent loss of committed work across restart, and idempotent duplicate handling.

The runtime architecture is illustrated in Fig. 1. Layers 1–4 sit on top of a common Apache Kafka event bus; layers 5 (persistence), 6 (model governance), and the system plane reside below. Agents only communicate via Kafka: Layer 1 deals with customers, invoices, payments, and behavioral indicators; Layer 2 uses financial APIs and litigation information to enrich data and aggregate signals; Layer 3 calculates payment risk and updates the compound score; Layer 4 performs credit policy and collections actions; Layer 5 keeps track of system state; Layer 6 measures drift and assesses challenger models; the system plane manages time ticks, monitoring, self-healing, DLQ monitoring, and runtime placement.

A. Agent Inventory and Topic Wiring

Table II catalogues every agent across the processing layers, governance plane, and system plane, summarizing its layer, topic subscriptions, publications, and operational role.

As detailed in Table II, each agent operates as an isolated OS process with a dedicated consumer group. The cross-cutting system plane manages heartbeats, runtime discovery, and poison-message isolation via dead-letter queues.

Architectural Rationale: Kafka-Native Actor Model vs. External Stream Engines. A central design decision in ACIS-X is using a unified Kafka event log rather than introducing external distributed stream-compute clusters (e.g., Apache Flink). While dedicated streaming engines provide distributed state backends, they also introduce additional operational complexity, synchronization overhead between clusters, and higher total cost of ownership (TCO). ACIS-X avoids these requirements by using Kafka partition keying: all business events are strictly keyed by

customer_id, ensuring that events for the same customer are routed to the same partition and processed in temporal order. This supports low-contention, per-customer in-memory state tracking within individual agent instances without requiring distributed join coordinators. Local state is periodically persisted to SQLite or PostgreSQL, while committed Kafka offsets enable deterministic recovery after system failures.

IV. Methodology

A. Three-Tier Signal Fusion

Tier 1 – Behavioural signals. The customer state agent rebuilds metrics from invoices and payments under a per-customer lock. Let $\mathcal{I}(c_i)$ be customer c_i ’s invoice set, I_k an invoice, and $\mathcal{P}$ the payment set. With payment/due timestamps $\text{paid}_k, \text{due}_k$, on-time ratio (OTR) and average delay are:

$$\begin{aligned} \text{OTR}(c_i) &= \frac{|\{I_k : \text{paid}(I_k) \leq \text{due}(I_k)\}|}{|\mathcal{I}(c_i)|}, \\ \text{AvgDelay}(c_i) &= \frac{1}{|\mathcal{P}|} \sum_k \max((\text{paid}_k - \text{due}_k), 0). \end{aligned} \quad (2)$$

Tier 2 – External signals. The external data agent caches financial ratios for 90 days, mapping them to financial risk $\mathcal{E}(c_i) \in [0, 1]$. Litigation scraping scores court exposure over 365 days as $\mathcal{L}(c_i) \in [0, 1]$.

Tier 3 – Fusion. The aggregator agent fuses external signals as:

$$R_{\text{ext}}(c_i) = w_{\text{fin}} \cdot \mathcal{E}(c_i) + w_{\text{lit}} \cdot \mathcal{L}(c_i) \in [0, 1], \quad (3)$$

with baseline weights $w_{\text{fin}} = 0.60$ and $w_{\text{lit}} = 0.40$, treating fundamentals as the primary baseline and litigation as a severe but sparser supplement. (We evaluate sensitivity to these weights in Section VI-B).

B. Payment Risk Prediction and Regulatory Guardrails

The payment prediction agent scores each open invoice. Feature vector $X \in \mathbb{R}^d$ encompasses normalised delay, credit utilisation, invoice ratio, late rate, external/litigation terms, and rating penalty. A Random Forest ($M = 50$ trees) yields base probability $r_{ml} = \frac{1}{M} \sum_{m=1}^M h_m(X)$:

$$r_{\text{final}} = \max(0, \min(1, r_{ml} + r_{\text{policy}})). \quad (4)$$

Here r_{policy} represents a deterministic regulatory guardrail: rating penalties (+0.10 for B, +0.20 for C) plus litigation adjustments (+0.15 if $\mathcal{L} > 0.1$, +0.25 if $\mathcal{L} > 0.5$). In enterprise credit governance, rare but catastrophic tail risks (e.g., active insolvency litigation) must not be smoothed out by statistical ensembles. Decoupling statistical r_{ml} from non-negotiable policy constraints r_{policy} guarantees compliance with credit risk management policies.

For auditability, we persist exact SHAP attributions [6] computed in polynomial time via TreeExplainer: $f(X) =$

TABLE I
Architectural positioning across enterprise credit risk paradigms

Dimension	Batch XAI [4]	Stream Risk [8]	Cognitive MAS [2], [11]	ACIS-X (Ours)
Latency Profile	Hours – Days	Sub-second (<1 s)	Multi-second (>2 s)	Sub-50 ms (P95 16 ms)
Signal Scope	Financial ratios	Single transaction stream	Text / financial reports	3-tier (AR + Fin + Legal)
Execution Model	Periodic batch scripts	Stream processing	Deliberative LLM / MARL	Reactive event-driven MAS
Atomicity Guarantee	DB ACID transactions	At-least-once / EOS	None (stateless LLM)	Native Kafka EOS + PK
Fault Recovery	Cold process restart	Node failover	None / prompt retry	Heartbeat + offset resume
Drift Governance	Offline re-training	External metric dashboard	Manual prompt tuning	Streaming PSI + Challenger

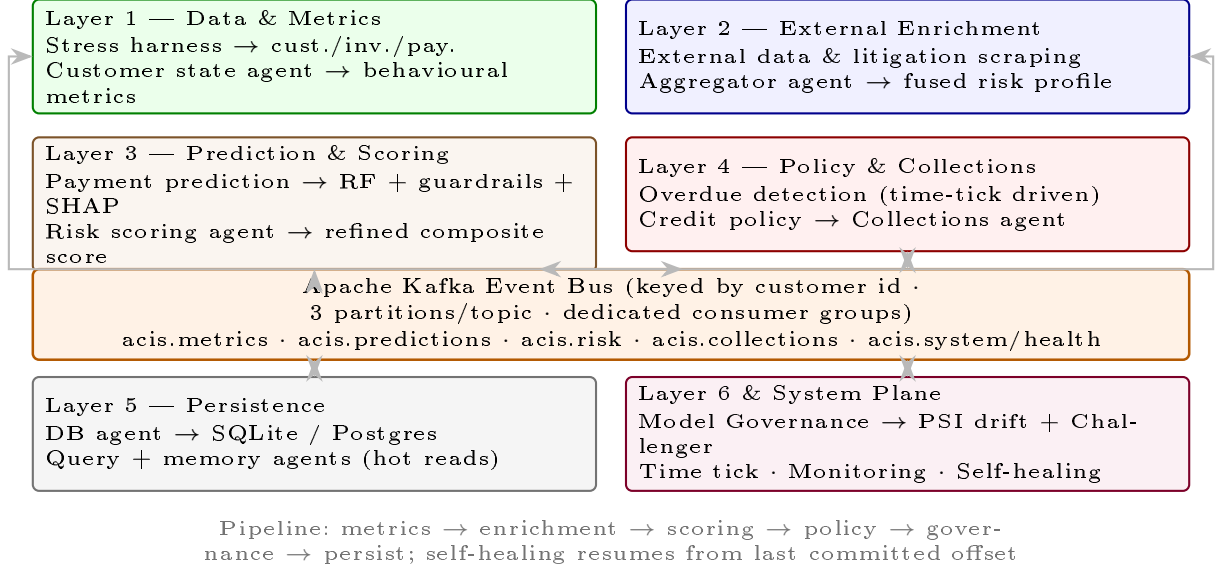


Fig. 1. ACIS-X layered architecture. Specialized reactive agents communicate exclusively through Kafka; the system plane manages health, discovery, and recovery.

TABLE II
ACIS-X Unified Agent Inventory: Layer, topic routing, and operational role

Agent	L	Subscribes To	Publishes To	Role / Operational Rationale
ScenarioGenerator	1	acis.commands	cust., inv., pay.	Emits reproducible synthetic and jump-diffusion macro-stress streams.
CustomerState	1	invoices, payments	acis.metrics	Rebuilds behavioural metrics (OTR, delay, utilisation).
ExternalData	2	acis.metrics	acis.metrics	Enriches and caches financial ratios (D/E, ROE, PE).
LitigationScraping	2	acis.metrics	acis.metrics	Quantifies legal exposure from court/news feeds.
Aggregator	2	acis.metrics	acis.risk	Fuses Tier 2 financial and litigation signals (0.6/0.4).
PaymentPrediction	3	acis.metrics	acis.predictions	Random Forest scoring + guardrails + SHAP attributions.
RiskScoring	3	pred., cust., met., risk	acis.risk	Refines base score with behavioural & temporal velocity.
OverdueDetection	4	acis.time	acis.invoices	Time-tick driven detection of delinquency freshness.
CreditPolicy	4	acis.risk	acis.collections	Maps risk tiers to operational control intents.
Collections	4	acis.collections	acis.collections	Executes idempotent collection actions without duplicate storms.
DBAgent	5	ALL topics	SQLite / Postgres	Persists events for state recovery across process restarts.
Query / Memory	5	(synchronous)	(in-memory cache)	Serves hot in-memory reads with SQLite/Postgres fallback.
ModelGovernance	6	pred., metrics	acis.system, health	Tracks PSI/KS drift and benchmarks challenger models.

$E[f(X)] + \sum_{j=1}^d \phi_j$. Policy additives sit outside this decomposition and are logged alongside the SHAP vector for complete mathematical transparency.

C. Risk Refinement and Self-Healing Protocol

The risk scoring agent forms the adjusted risk r_{adj} from prediction r_{pred} :

$$r_{\text{adj}} = \max\left(0, \min\left(1, r_{\text{pred}} + 0.85\Delta_{\text{beh}} + 0.30\Delta_{\text{temp}}\right)\right), \quad (5)$$

where $\Delta_{\text{beh}} = 0.40a_{\text{overdue}} + 0.20a_{\text{payment}} + 0.15a_{\text{delay}} + 0.15a_{\text{outstanding}} + 0.10a_{\text{exposure}}$ ranks operational urgency

across normalised factors $a. \in [0, 1]$. Temporal velocity $\Delta_{\text{temp}} \in [-0.2, 0.3]$ captures trend direction.

Self-Healing Protocol. Agents emit heartbeats every $T_H = 10$ s. Monitoring detects a timeout after 60s of silence (6 missed beats), triggering an event-driven restart request to resume consumer execution from the last committed offset.

D. Autonomous Model Governance and Challenger Benchmarking

To support model-risk governance principles aligned with SR 11-7, Layer 6 introduces the ModelGovernanceAgent. It tracks streaming predictions across sliding windows ($W = 200$) and computes the Population Stability Index (PSI):

$$\text{PSI} = \sum_{b=1}^B (P_b - Q_b) \cdot \ln \left(\frac{P_b}{Q_b} \right), \quad (6)$$

where P_b and Q_b are baseline and current empirical quantile proportions across $B = 10$ bins. Values $\text{PSI} \geq 0.10$ trigger automated drift warnings. In parallel, an online challenger model (linear SGD classifier) is continuously trained and benchmarked against the champion Random Forest, publishing periodic audit reports to acis.system.

E. Jump-Diffusion Synthetic Stress Test Harness

To evaluate ACIS-X across diverse macroeconomic stress cases without exposing confidential commercial ledgers, we implement a jump-diffusion synthetic stress test harness:

$$dZ_t = \kappa(\mu - Z_t)dt + \sigma_Z dW_t + J_t dN_t, \quad (7)$$

where $Z_t \in [0, 1]$ is a latent enterprise stress factor, κ the mean-reversion rate, σ_Z diffusion volatility, and $J_t dN_t$ a Poisson jump process representing liquidity shocks. The harness conditions customer delinquency on Z_t via fat-tailed Pareto delay mixtures across Calm, Stressed, and Crisis regimes. This jump-diffusion harness is used to generate diverse and reproducible stress scenarios for architectural and comparative evaluation, not to establish production-level statistical fidelity.

V. Implementation

The solution is implemented in Python 3.11 across ~ 55 files within seven packages: agents, governance, runtime, self-healing, monitoring, registry, and schemas. The entry point creates Kafka topics in advance and launches agents as separate OS processes managed by supervisor, with a maximum of 5 attempts in 2 minutes. Messages use a Pydantic-validated schema (v1.1) containing the correlation ID; validation failures are routed to a dead letter queue.

EOS for Native Kafka. Transport-level de-duplication uses Kafka idempotent producers, all-in-sync-replica acknowledgments, and bounded in-flight requests. Consumers use committed read isolation and consume only committed transaction records. When transactional processing mode is enabled, Kafka transactions ensure atomicity for consume-transform-produce operations through transactional offset commits, including across topic boundaries. Downstream in-memory LRU filtering and database uniqueness constraints on the primary key provide additional defense-in-depth protection against consumer reconnection or replay. This design preserves reliable message

delivery and strengthens recovery guarantees while keeping processing semantics consistent across failures, retries, and replays.

VI. Experimental Evaluation

A. Test Infrastructure and Systems Metrics

The test suite executed on Windows 11 (i7-12700H, 32 GB) with Docker Kafka active: 180 passed (100%), 0 failed, across unit, integration, governance, and fault tests. Table III reports key systems metrics.

TABLE III
Systems metrics from the automated suite

Metric	Value	Source
Pipeline P95 latency (mocked, $n = 500$)	16 ms (mean 4.6 ms)	performance suite
Collections dedup under 10k repeats	1 action (perfect dedup)	performance suite
Risk-scoring scaling	3.49x at $N=4$ (4.93x at $N=6$)	throughput test
Model governance PSI calculation	< 0.10 stable, ≥ 0.25 drift	governance suite
Suite result	180/180 pass (100%)	complete test suite

As shown in Table III, pipeline P95 latency remains well within sub-50 ms streaming targets, while horizontal scaling demonstrates near-linear throughput increases across dedicated consumer partitions.

B. Architectural Ablation and Sensitivity Analysis

To quantify component contributions, we performed an architectural ablation on $n=200$ synthetic customers across 5 random seeds (42, 123, 456, 789, 1024). Table IV presents Spearman rank correlation (ρ) with ground-truth risk (mean $\pm$ std).

TABLE IV
Architectural ablation ($n=200$, 5 seeds). ρ = Spearman rank correlation with ground truth (mean $\pm$ std).

Configuration	ρ	$\Delta\rho$	Note
Full ACIS-X (baseline)	0.713 ± 0.024	—	all layers active
—External enrichment	0.711 ± 0.027	-0.002	Tier 2/3 signals ablated
—Behavioural refinement	0.653 ± 0.033	-0.060	raw utilisation only
—SHAP audit layer	(latency)	$+0.9$ ms/event	explainability overhead

Table IV demonstrates that behavioural refinement provides a substantial ranking lift across the five-seed evaluation ($\Delta\rho = +0.060$, paired $t = 10.47$, $p < 0.001$), while external enrichment’s marginal contribution ($\Delta\rho = +0.002$, $p = 0.27$) is small and not clearly distinguishable from noise at $n = 200$ across 5 seeds. This indicates that real-time behavioural transaction dynamics are the primary operational driver of intraday credit differentiation, with external ratios acting primarily as secondary policy guardrails. SHAP TreeExplainer adds merely ~ 0.9 ms per event.

Weight Sensitivity Analysis. To evaluate the sensitivity of the 0.60/0.40 fusion heuristic, we evaluated weight splits across 50/50, 60/40, and 70/30 ($w_{\text{fin}}/w_{\text{lit}}$) over 5 seeds. Ranking correlation with ground truth remains positive across all configurations ($\rho_{50/50} = 0.761 \pm 0.004$, $\rho_{60/40} = 0.873 \pm 0.004$, $\rho_{70/30} = 0.941 \pm 0.002$), while increasing with financial weight w_{fin} . This upward trend is consistent with the synthetic ground-truth risk function (Eq. 1), which incorporates external financial health

alongside behavioural delinquency. Rather than asserting model-free optimality, this analysis demonstrates that the fusion pipeline responds predictably to signal weighting within the tested synthetic scenarios without requiring complex online reinforcement learning.

We evaluated the self-healing plane by injecting controlled faults into the SelfHealingAgent (Table V).

TABLE V

Fault injection results (mocked Kafka, single-node decision latency).

Scenario	Metric	Value	Assertion
Agent hard crash (TIMEOUT)	decision latency	0.26 ms	<100 ms
Latency spike (5 s, CPU 95%)	decision latency	0.10 ms	<100 ms
Cascading 5-agent failure	restarts/agent	1	exactly 1 (storm prevention)
100-cycle endurance	success rate	100%	100/100
100-cycle endurance	P95 decision latency	0.23 ms	<50 ms

Table V shows that self-healing decision latency is sub-millisecond (0.26 ms for hard crashes under single-node conditions). The cooldown mechanism provides storm prevention (1 restart per agent during 5-agent cascading failure) with 100% success across 100 endurance cycles (P95 decision latency 0.23 ms).

C. Predictive Comparison and Methodological Nuance

To evaluate risk scoring capabilities, we benchmark ACIS-X against naive heuristics and conventional supervised learners on a synthetic hold-out dataset (Table VI, $n = 500$, 70/30 split, mean $\pm$ std across 5 seeds: 42, 123, 456, 789, 1024).

TABLE VI

Predictive benchmark on synthetic hold-out ($n = 500$, 5 seeds, mean $\pm$ std). Supervised models are trained directly on generator ground-truth labels; ACIS-X operates label-free.

Method	F1	AUC
Naive utilisation threshold	0.730 $\pm$ 0.034	—
ACIS-X streaming refinement (label-free)	0.758 $\pm$ 0.016	0.862 $\pm$ 0.025
Logistic Regression (basic / enriched, supervised)	0.929 $\pm$ 0.027 / 0.967 $\pm$ 0.017	0.988 $\pm$ 0.006 / 0.997 $\pm$ 0.002
Random Forest (basic / enriched, supervised)	0.889 $\pm$ 0.024 / 0.895 $\pm$ 0.029	0.970 $\pm$ 0.013 / 0.975 $\pm$ 0.011
Gradient Boosting (basic / enriched, supervised)	0.869 $\pm$ 0.032 / 0.887 $\pm$ 0.039	0.965 $\pm$ 0.012 / 0.970 $\pm$ 0.013

Table VI illustrates an essential architectural distinction: conventional supervised models (Logistic Regression, Random Forest, GBDT) achieve high metrics (AUC up to 0.997) because they are trained with full supervision directly on the synthetic generator’s mathematical labelling function. In contrast, ACIS-X operates as an online, label-free streaming framework that requires no historical default training labels. In this label-free setting, ACIS-X multi-signal refinement yields a modest mean F1 improvement over the naive heuristic rule (0.758 $\pm$ 0.016 vs 0.730 $\pm$ 0.034, winning in 4/5 seed trials, paired $t = 1.42$, $p = 0.23$) while critically providing continuous probability ranking and temporal velocity (AUC 0.862 $\pm$ 0.025) that single-threshold heuristics cannot generate. Supervised models confirm that external features provide strong discriminative signal when explicit default labels are available.

VII. Discussion and Limitations

The ablation study (Table IV) shows that behavioral refinement is the key factor behind improved ranks

($\Delta\rho = 0.060$, $p < 0.001$), while SHAP explainability adds less than 1 ms per event. The fault injection study (Table V) demonstrates sub-millisecond self-healing decision latency under single-node conditions without restart storms. This metric measures decision latency and restart dispatch latency; in production distributed environments, full recovery latency will also depend on consumer group rebalance timeout (3 to 10 seconds). The governance plane further shows that real-time PSI and KS tests can run online alongside event-based pipelines.

Limitations and Domain Context. We explicitly acknowledge four core limitations: (1) Synthetic vs. real data: synthetic stochastic generation models stylized delinquency dynamics but cannot fully replicate non-linear macroeconomic contagions or strategic invoice deferrals; (2) Evaluation scope: results represent architectural and systems validation (latency, scaling, self-healing, idempotency, and fusion) under controlled conditions; (3) Production readiness: deployment requires empirical backtesting on enterprise ERP ledgers and multi-datacenter disaster recovery drills; and (4) Sample size of multi-seed trials: paired significance tests use $n = 5$ random seeds; while ranking trends are consistent across seeds, larger seed sets ($n \geq 30$) will provide greater statistical power. Furthermore, the 60-second heartbeat timeout is tailored for B2B trade credit (30–60 day terms), where a 60-second window provides near-instantaneous intraday response compared to 24–168 hour batch cycles, while suppressing failover flapping from transient GC pauses.

VIII. Conclusion

ACIS-X is a Kafka reactive multi-agent credit pipeline featuring three-tier fusion, native EOS transport guarantees, autonomous model governance, SHAP audit persistence, and heartbeat-driven recovery. On the automated test suite, all 180 tests passed; pipeline P95 latency reached 16 ms and scaling reached 3.49 $\times$ at 4 workers. Architectural ablation confirmed behavioural refinement delivers the largest rank lift ($\Delta\rho = 0.060$) with <1 ms SHAP overhead, while Sensitivity analysis showed a predictable increase in ranking correlation with financial-signal weight within the tested synthetic scenarios. Fault injection demonstrated sub-millisecond self-healing decision latency under controlled conditions with 100% endurance success. Future work includes empirical validation on enterprise ERP ledgers and multi-cluster disaster recovery drills.

Acknowledgment

The authors thank the faculty and laboratory infrastructure of the Department of Computer Science and Engineering, AMC Engineering College, Bengaluru, for supporting this research.

References

- [1] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in Proc. NetDB, 2011.

- [2] G. Jajoo, A. Pandey, P. Chitale, and S. Agarwal, "MASCA: LLM-based multi-agent system for credit assessment," arXiv preprint arXiv:2507.22758, 2025.
- [3] E. I. Altman, "Financial ratios, discriminant analysis and the prediction of corporate bankruptcy," *J. Finance*, vol. 23, no. 4, pp. 589–609, 1968.
- [4] V. Chang, Q. A. Xu, S. H. Akinloye, V. Benson, and K. Hall, "Prediction of bank credit worthiness through credit risk analysis: an explainable machine learning study," *Ann. Oper. Res.*, vol. 354, pp. 247–271, 2025.
- [5] C. N. Nwafor, O. Nwafor, and S. Brahma, "Enhancing transparency and fairness in automated credit decisions: an explainable novel hybrid machine learning approach," *Sci. Rep.*, vol. 14, p. 25174, 2024.
- [6] S. M. Lundberg, G. Erion, H. Chen, et al., "From local explanations to global understanding with explainable AI for trees," *Nat. Mach. Intell.*, vol. 2, no. 1, pp. 56–67, 2020.
- [7] S. Kesarpu and H. P. Dasari, "Kafka event sourcing for real-time risk analysis," *Int. J. Comput. Exp. Sci. Eng.*, vol. 11, no. 3, 2025.
- [8] H. Parnerkar, "Comparative study of event-driven architectures in financial systems for real-time risk analysis and mitigation," *Eur. J. Electr. Eng. Comput. Sci.*, vol. 9, no. 6, pp. 46–54, 2025.
- [9] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.
- [10] K. R. Beerreddy, "Self-healing streaming architecture: resilience patterns for event-driven microservices," *J. Comput. Sci. Technol. Stud.*, vol. 7, no. 8, pp. 254–268, 2025.
- [11] A. Kubam, "Agentic AI for autonomous, explainable, and real-time credit risk decision-making," arXiv preprint arXiv:2601.00818, 2026.
- [12] G. Kou et al., "Learning to fuse: cost-sensitive credit assessment via hierarchical multi-agent reinforcement learning," in *Proc. NeurIPS GenAI in Finance*, 2025.